BUỔI 12 – PHÂN CÔNG NGHIÊN CỨU LÝ THUYẾT

Modular Monolithic – Tư duy thiết kế và con đường tiến hóa lên Microservice

Giai đoạn hiện tại: Chỉ nghiên cứu lý thuyết và chuẩn bị slide. Chưa triển khai demo, chưa viết code, chưa cài RabbitMQ.

1. Mục tiêu và nguyên tắc phân công

Phân chia cho 3 thành viên theo ba mảng chuyên môn, bảo đảm mỗi người có nội dung nghiên cứu chính, đầu ra slide rõ ràng và hạn chế trùng lặp. Nội dung bám theo tài liệu Buổi 12, các mục 2.1–2.6.

Thành viên	Mảng phụ trách	Phạm vi
TV1	Kiến trúc và tiến hóa	Mục 2.1, 2.3
TV2	Phân rã nghiệp vụ và thiết kế module	Mục 2.2
TV3	Giao tiếp đồng bộ / bất đồng bộ	Mục 2.4, 2.5, 2.6

2. TV1 – Kiến trúc và con đường tiến hóa

Phạm vi: Mục 2.1 và 2.3

Phần 1 – So sánh kiến trúc

• Monolithic là gì?
• Microservices là gì?
• Modular Monolithic là gì?
• Điểm giống và khác nhau giữa ba kiến trúc.
• Ưu điểm, nhược điểm và lý do Modular Monolithic nằm giữa hai mô hình còn lại.

Phần 2 – Tiêu chí lựa chọn

• Quy mô đội ngũ.
• Nguồn lực hạ tầng và DevOps.
• Giai đoạn phát triển sản phẩm.
• Vì sao đội nhỏ thường phù hợp Monolithic/Modular Monolithic; vì sao Microservices làm tăng độ phức tạp vận hành; khi nào có lý do tách service.

Phần 3 – MonolithFirst

• Không nhất thiết bắt đầu bằng Microservices.
• Bắt đầu với Modular Monolithic.
• Chỉ tách module khi có lý do thực tế.
• Không cần chuyển toàn bộ hệ thống cùng lúc; có thể tồn tại kiến trúc Hybrid.

Phần 4 – Lộ trình tiến hóa

1. Tách schema/database của module.
2. Thay interface nội bộ bằng giao tiếp qua network.
3. Đóng gói module thành Docker image riêng.
4. Tạo pipeline CI/CD riêng.

Đầu ra: khoảng 5–6 slide về kiến trúc, tiêu chí lựa chọn, MonolithFirst và lộ trình 4 bước.

3. TV2 – Tư duy phân rã nghiệp vụ

Phạm vi: Mục 2.2 – phần trọng tâm của buổi học

Phần 1 – Domain Decomposition

• Domain Decomposition là gì?
• Vì sao cần chia nghiệp vụ thành module?
• Vì sao không nên chia module theo cảm tính?
• Mối liên hệ giữa nghiệp vụ và ranh giới module.

Phần 2 – Bounded Context

• Bounded Context là gì?
• Ngữ cảnh nghiệp vụ riêng và ngôn ngữ nghiệp vụ riêng.
• Vì sao module hạn chế phụ thuộc vào chi tiết bên trong module khác?

Phần 3 – Bộ 4 câu hỏi xác định module

1. Nghiệp vụ có ngôn ngữ và khái niệm riêng không?
2. Bộ phận nào chịu trách nhiệm nghiệp vụ trong tổ chức?
3. Dữ liệu có thể đứng độc lập, không cần JOIN xuyên module không?
4. Nghiệp vụ có tốc độ thay đổi khác biệt không?

Phần 4 – Phân rã E-commerce

Nghiên cứu vai trò của Catalog, Cart, Order, Payment, Inventory, Shipping, Identity, Notification, Promotion và Review; giải thích vì sao chúng có thể được tổ chức thành các module khác nhau.

Phần 5 – Cấu trúc module và cô lập

• Project/thư mục riêng.
• Layered/Clean Architecture bên trong module: Domain, Application, Infrastructure.
• Schema/database riêng.
• Không truy cập Repository/DbContext của module khác.
• Giao tiếp qua Public Interface.

Đầu ra: khoảng 6–7 slide về domain, Bounded Context, bộ 4 câu hỏi, 10 nghiệp vụ và nguyên tắc module độc lập.

4. TV3 – Giao tiếp đồng bộ và bất đồng bộ

Phạm vi: Mục 2.4, 2.5 và 2.6

Phần 1 – Giao tiếp đồng bộ

• Synchronous Communication và cơ chế gọi/chờ phản hồi.
• RESTful API và gRPC.
• Điểm khác nhau, ưu/nhược điểm và tình huống sử dụng.

Phần 2 – Rủi ro đồng bộ

• Gọi đồng bộ nhiều tầng.
• Service chậm hoặc sập.
• Cascading Failure và ảnh hưởng đến toàn bộ luồng nghiệp vụ.

Phần 3 – Giao tiếp bất đồng bộ

• Asynchronous Communication và Message Queue.
• Producer/Publisher, Consumer/Subscriber, Queue, Exchange, Binding, Routing Key.
• Direct, Topic và Fanout Exchange.
• Publish/Subscribe.

Phần 4 – RabbitMQ trong E-commerce

Nghiên cứu ví dụ OrderCreated: Order phát event; Inventory, Notification và Promotion nhận event. Giải thích vì sao các module xử lý độc lập và Order không cần chờ toàn bộ Consumer.

Phần 5 – Chọn Sync hay Async

• Cần phản hồi ngay để quyết định → đồng bộ.
• Chỉ cần thông báo sự kiện → bất đồng bộ.
• Ví dụ: Order kiểm tra tồn kho trước khi xác nhận; sau khi tạo đơn, phát event cho các nghiệp vụ xử lý tiếp.

Đầu ra: khoảng 6–7 slide về Sync, REST/gRPC, Cascading Failure, RabbitMQ và nguyên tắc chọn Sync/Async.

5. Bảng phân công nghiên cứu

Nội dung	TV1	TV2	TV3
So sánh 3 kiến trúc	Chủ trì		
Tiêu chí chọn / MonolithFirst	Chủ trì		
Lộ trình tiến hóa Microservices	Chủ trì		
Domain Decomposition		Chủ trì	
Bounded Context		Chủ trì	
Bộ 4 câu hỏi phân rã		Chủ trì	
10 nghiệp vụ E-commerce		Chủ trì	
Schema riêng / Public Interface		Chủ trì	
REST vs gRPC			Chủ trì
Cascading Failure			Chủ trì
RabbitMQ / Exchange / Queue / Binding			Chủ trì
Publish/Subscribe			Chủ trì
Nguyên tắc Sync/Async			Chủ trì

6. Quy định chung khi nghiên cứu

Mỗi thành viên cần:
1. Đọc đúng phần được phân công trong tài liệu Buổi 12.
2. Hiểu và giải thích được khái niệm.
3. Tự tổng hợp bằng lời của mình.
4. Chuẩn bị slide theo phạm vi đã giao.
5. Có ví dụ minh họa E-commerce phù hợp.
6. Chuẩn bị câu hỏi giảng viên có thể hỏi.
7. Gửi nội dung cho nhóm trưởng kiểm tra.

Chưa thực hiện ở giai đoạn này: Không dựng project, viết code, tạo database, cài RabbitMQ, viết Producer/Consumer, triển khai REST/gRPC, chạy Docker demo hoặc phân chia nhiệm vụ demo.

7. Thứ tự ghép slide

1. TV1 trình bày tổng quan kiến trúc và tiêu chí lựa chọn.
2. TV2 trình bày tư duy phân rã nghiệp vụ và thiết kế module.
3. TV3 trình bày giao tiếp đồng bộ, bất đồng bộ và nguyên tắc lựa chọn.
4. Cả nhóm tổng kết mối liên hệ giữa kiến trúc, module và giao tiếp.

8. Kết luận

Phân công theo ba mảng: TV1 – Kiến trúc và tiến hóa; TV2 – Phân rã domain và thiết kế module; TV3 – Giao tiếp đồng bộ, bất đồng bộ và RabbitMQ. Giai đoạn này chỉ hoàn thành nghiên cứu và slide. Sau khi thống nhất lý thuyết, nhóm mới chuyển sang lập kế hoạch demo.

Tài liệu căn cứ: “Buổi 12 Modular Monolithic – Tư duy thiết kế và con đường tiến hóa lên Microservice” và “Kế Hoạch Phân Công Nghiên Cứu Lý Thuyết Buổi 12”.